1. Course Enrollment Management System

Develop a Course Enrollment Management System using the MERN Stack.

MongoDB Collection: Courses

Using Mongoose, create a collection named courses with the following fields:

- courseId
- courseName
- instructor
- duration
- fees

Backend (ExpressJS APIs)

Develop RESTful API endpoints using ExpressJS to perform the following operations:

1. Add Course: Create a new course and store its details in MongoDB.
2. Fetch Courses: Retrieve and display all available courses.
3. Update Course: Update the duration or fees of an existing course using its ID.

Frontend (ReactJS)

Develop the following pages using ReactJS and implement React Router for navigation between pages.

1. Home Page: Display the system title and a brief introduction to the Course Enrollment Management System.
2. Add Course Page: Create a form to enter course details and submit the form to add a new course.
3. View Courses Page: Fetch and display all courses in a table with appropriate details.
4. Edit Course Page: Provide an Update button against each course in the table. On clicking the button, navigate to a separate edit page with the selected course details pre-filled in the form. Allow the user to update the course duration or fees.
5. Navigation: Implement React Router to navigate between Home, Add Course, and View Courses pages.
6. Redirection: After successfully updating a course, redirect the user to the View Courses page and display the updated course details.

2. Hotel Room Booking System

MongoDB Collection: Rooms

Using Mongoose, create a collection named rooms with the following fields:

- roomType
- pricePerNight
- status (Available / Booked)

Backend (ExpressJS APIs)

1. Add a new room.

2. Fetch all rooms.
3. Update the room status to "Booked" using its ID.

Frontend (ReactJS)

Develop the following pages using React Router:

1. Home Page: Display the hotel name and a brief overview of its facilities and services.
2. Add Room Page: Create a form to add new room details.
3. View Rooms Page: Fetch and display all rooms in a table.
4. Filter Rooms: Provide a dropdown to filter rooms by status (Available / Booked). Implement filtering on the frontend only.
5. Book Room: Provide a Book button against each available room. On clicking it, update the room status to "Booked" by calling the backend API.
6. Navigation: Implement React Router to navigate between Home, Add Room, and View Rooms pages.

3. Library Member Registration Portal

MongoDB Collection: Members

Using Mongoose, create a collection named members with the following fields:

- fullName
- membershipType
- joinDate

Backend (ExpressJS APIs)

1. Add a new member.
2. Fetch all registered members.
3. Delete a member using their ID.

Frontend (ReactJS)

Develop the following pages using React Router:

1. Home Page: Display information about the library, its facilities, and membership services.
2. Add Member Page: Create a form to register a new library member.
3. View Members Page: Fetch and display all registered members in a table.
4. Delete Member: Provide a Delete button against each member to remove their record from the database.
5. Navigation: Implement React Router to navigate between Home, Add Member, and View Members pages.

4. Sports Tournament Team Management System

MongoDB Collection: Teams

Using Mongoose, create a collection named teams with the following fields:

- teamId
- teamName
- coach
- category (Junior / Senior)
- playersCount

Backend (ExpressJS APIs)

1. Add a new team.
2. Fetch all registered teams.

Frontend (ReactJS)

Develop the following pages using React Router:

1. Home Page: Display the tournament name and a brief introduction.
2. Add Team Page: Create a form to register a new team.
3. View Teams Page: Fetch and display all teams in a table.
4. Search Teams: Provide a dropdown to search/filter teams by category (Junior / Senior). Implement filtering on the frontend only.
5. Navigation: Implement React Router to navigate between Home, Add Team, and View Teams pages.

5. Driver Management System

MongoDB Collection: Drivers

Using Mongoose, create a collection named drivers with the following fields:

- name
- licenseNumber
- experienceYears
- contact

Backend (ExpressJS APIs)

1. Add a new driver.
2. Fetch all registered drivers.
3. Delete a driver using their ID.

Frontend (ReactJS)

Develop the following pages using React Router:

1. Home Page: Display the system title and a brief introduction to the driver management system.
2. Add Driver Page: Create a form to register a new driver.
3. All Drivers Page: Fetch and display all drivers in a table.
4. Delete Driver: Provide a Delete button against each driver to remove their record from the database.

5. Navigation: Implement React Router to navigate between Home, Add Driver, and All Drivers pages.

6. Hostel Student Management System

MongoDB Collection: HostelStudents

Using Mongoose, create a collection named hostelStudents with the following fields:

- name
- roomNo
- course
- contact

Backend (ExpressJS APIs)

1. Add a new hostel student.
2. Fetch all registered hostel students.

Frontend (ReactJS)

Develop the following pages using React Router:

1. Home Page: Display hostel information, facilities, and a brief introduction.
2. Add Student Page: Create a form to register a new hostel student.
3. View Students Page: Fetch and display all hostel students in a table.
4. Search Students: Provide a search box to filter students based on their room number. Implement searching on the frontend only.
5. Navigation: Implement React Router to navigate between Home, Add Student, and View Students pages.

7. Medicine Inventory Management System

MongoDB Collection: Medicines

Using Mongoose, create a collection named medicines with the following fields:

- name
- type
- stock
- price

Backend (ExpressJS APIs)

1. Add a new medicine.
2. Fetch all available medicines.

Frontend (ReactJS)

Develop the following pages using React Router:

1. Home Page: Display the system title and a brief introduction to medicine inventory management.

2. Add Medicine Page: Create a form to add new medicine details.
3. View Medicines Page: Fetch and display all medicines in a table.
4. Sort Medicines: Provide a dropdown to sort medicines by price (Low to High / High to Low). Implement sorting on the frontend only.
5. Navigation: Implement React Router to navigate between Home, Add Medicine, and View Medicines pages.

8. Photography Studio Booking System

MongoDB Collection: Bookings

Using Mongoose, create a collection named bookings with the following fields:

- customerName
- serviceType
- date
- amount

Backend (ExpressJS APIs)

1. Create a new booking.
2. Fetch all bookings.
3. Delete a booking using its ID.

Frontend (ReactJS)

Develop the following pages using React Router:

1. Home Page: Display the photography studio overview and list of services offered.
2. Add Booking Page: Create a form to enter customer and booking details.
3. View Bookings Page: Fetch and display all bookings in a table.
4. Delete Booking: Provide a Delete button against each booking to remove the booking from the database.
5. Navigation: Implement React Router to navigate between Home, Add Booking, and View Bookings pages.

9. Real Estate Property Listing System

MongoDB Collection: Properties

Using Mongoose, create a collection named properties with the following fields:

- title
- location
- price
- propertyType

Backend (ExpressJS APIs)

1. Add a new property.

2. Fetch all listed properties.

Frontend (ReactJS)

Develop the following pages using React Router:

1. Home Page: Display the real estate agency overview and a brief introduction to its services.
2. Add Property Page: Create a form to add new property details.
3. View Properties Page: Fetch and display all properties in a table.
4. Filter Properties: Provide a dropdown to filter properties by type. Implement filtering on the frontend only.
5. Sort Properties: Provide a dropdown to sort properties by price (Low to High / High to Low). Implement sorting on the frontend only.
6. Navigation: Implement React Router to navigate between Home, Add Property, and View Properties pages.

10. Food Delivery Menu Management System

MongoDB Collection: MenuItems

Using Mongoose, create a collection named menuItems with the following fields:

- itemName
- category
- price
- availability

Backend (ExpressJS APIs)

1. Add a new menu item.
2. Fetch all menu items.
3. Delete a menu item using its ID.

Frontend (ReactJS)

Develop the following pages using React Router:

1. Home Page: Display the food delivery service overview and a brief introduction to the available food categories.
2. Add Menu Item Page: Create a form to add new menu items.
3. View Menu Items Page: Fetch and display all menu items in a table.
4. Filter Menu Items: Provide a dropdown to filter menu items by category. Implement filtering on the frontend only.
5. Delete Menu Item: Provide a Delete button against each menu item to remove it from the database.
6. Navigation: Implement React Router to navigate between Home, Add Menu Item, and View Menu Items pages.

11. Job Application Tracker

MongoDB Collection: Applications

Using Mongoose, create a collection named applications with the following fields:

- applicantName
- position
- status
- email

Backend (ExpressJS APIs)

1. Add a new job application.
2. Fetch all job applications.
3. Update the status of an existing application using its ID.

Frontend (ReactJS)

Develop the following pages using React Router:

1. Home Page: Display the system title and a brief introduction to the job application tracking system.
2. Add Application Page: Create a form to submit a new job application.
3. View Applications Page: Fetch and display all job applications in a table.
4. Update Status: Provide an Update Status button against each application. On clicking it, allow the user to change the application status and save the updated status to the database.
5. Navigation: Implement React Router to navigate between Home, Add Application, and View Applications pages.